

DiracX developments: directions

Alexandre F. Boyer · CERN

12th DIRAC(X) Users' Workshop · FZU Prague · 13–16 October 2026

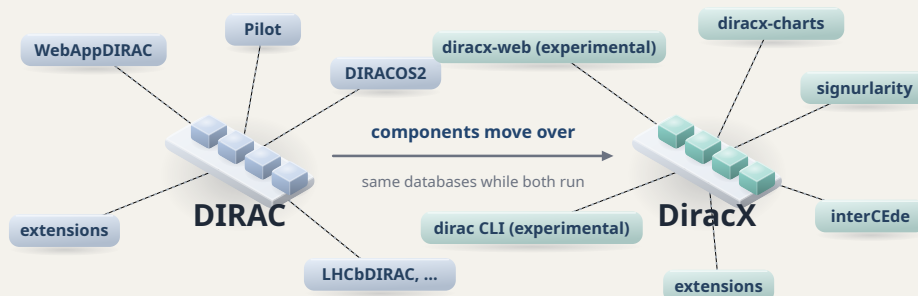


| 1 • Where we are now

DIRAC and DiracX today



Maintaining DIRAC, progressing on DiracX



DIRAC v9.1.21 · v9.0.27 · v8.0.85

WebAppDIRAC 6.0.6 · DIRACOS2 2.63

Patched, not developed: security fixes, and performance fixes when needed.

DiracX v0.6.0: foundations, authentication, jobs (search, status), sandboxes, read-only resource status, and experimental observability (OpenTelemetry).

The web app (diracx-web 0.1.0-a11) and the `dirac` CLI are still experimental.

Adopt what already exists in DiracX. DIRAC is only patched, so that DiracX can progress. LHCb runs one service and one task in production; two other installations are certifying DiracX.

2 · Evolving requirements, fruitful collaboration

New communities, new use cases, and two systems running side by side



What DIRAC is for, and what it values

Running large-scale workflows of embarrassingly parallel jobs on heterogeneous, distributed computing and storage resources: high-throughput computing.

Minimal dependencies

A database and a search index, on a few hosts.

In DiracX: standard protocols for what is external (S3, OpenTelemetry, OIDC), and options without Kubernetes.

Multi-VO installations

One installation serves several communities.

In DiracX: multi-VO from the start.

Extendable

Communities add their own pieces.

In DiracX: extensions, with a reference extension.

My own reading of what communities have asked for over the last few years.

Requirements keep evolving

New communities

CMS and FCC, who joined this summer, bring new requirements. CMS has no DIRAC installation and runs its own workload management.

Advanced needs, in extensions

CTAO, LHCb, Belle II and JUNO built extensions for advanced needs that are now close to each other. They belong in the common project: CTAO's use of **CWL** turned out useful for DiracX as a whole.

Constant needs

GridPP has few resources: DiracX must stay very simple to deploy, with minimal dependencies.

Share your needs early, at DOPs, before building around the system: someone else may need the same thing.

What stays stable, what is new

Stable

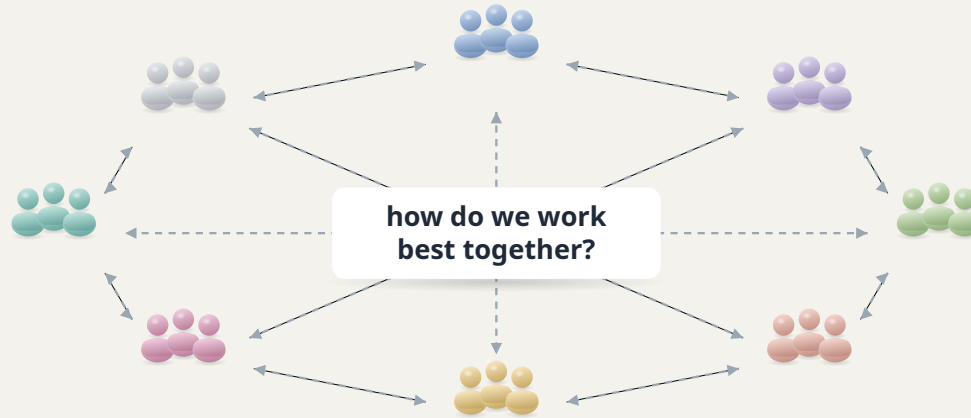
- **DIRAC keeps running**, with security and performance patches.
- Components move **one at a time**, over the same databases.
- **Extensions keep working** while components move.

New

- **The transformation system**: a new data model, new logic, and a much richer scope than DIRAC's.
- **Decoupled from DIRAC**, so DIRAC's databases need no large changes and stay stable.
- **Standard protocols** (sandboxes in S3, observability in OpenTelemetry) and multi-VO from the start.

A stable path for what exists, and room for what is new.

Many part-time hands



- **36 people** contributed since the last workshop, most of them at **20 to 30 %** of their time: about **4 to 5 full-time equivalents**.
- They maintain DIRAC and develop DiracX at the same time.

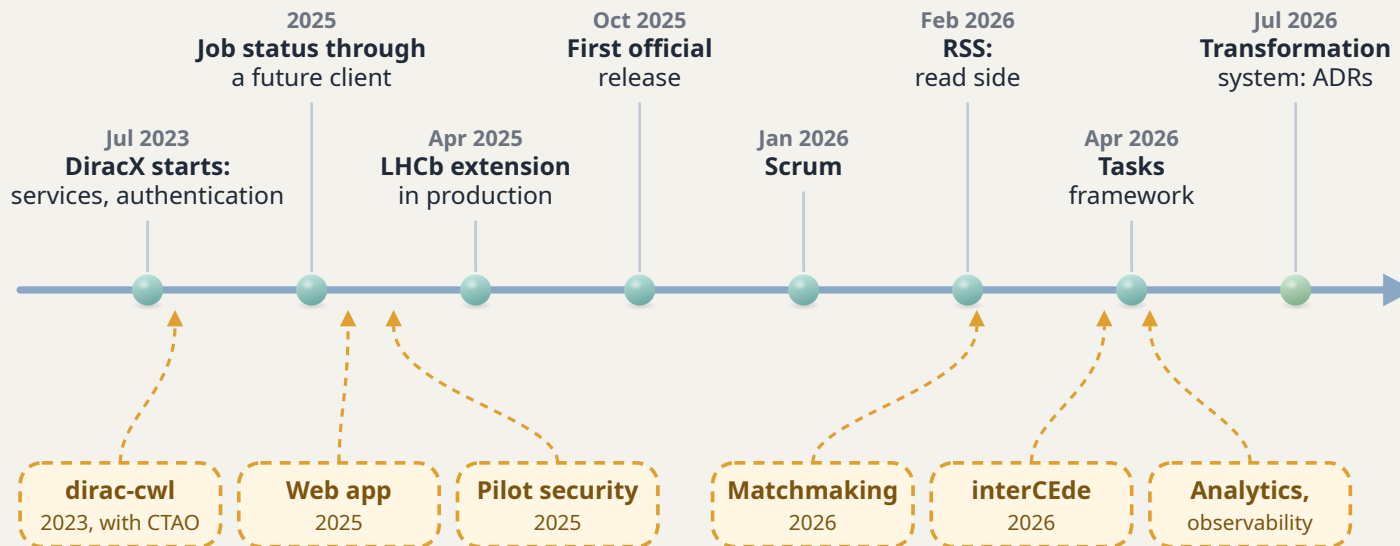
Finding the best way to work with each other is paramount.

3 · Adapting quickly

Responding to changing requirements, in small steps



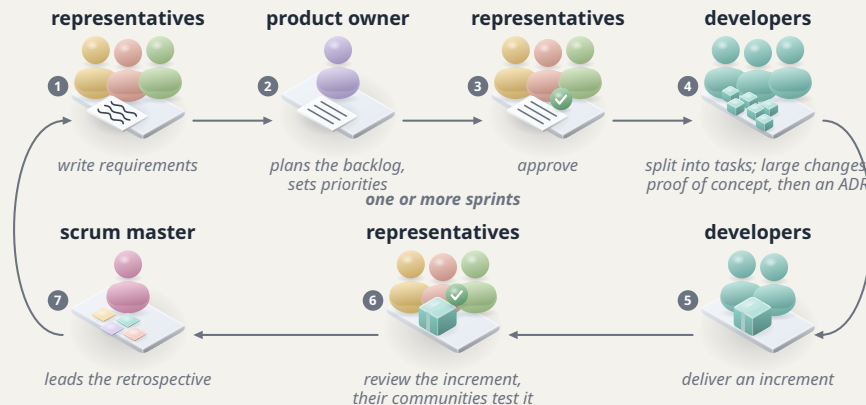
The journey so far



In parallel: proofs of concept, linked to when they started

- From 2023, three to four hackathons a year onboarded new contributors, with no dashboard or regular meeting to follow progress. Scrum brought both in 2026.
- Proofs of concept run in parallel, not mandatory: we try an idea before committing to it.

Almost the current process

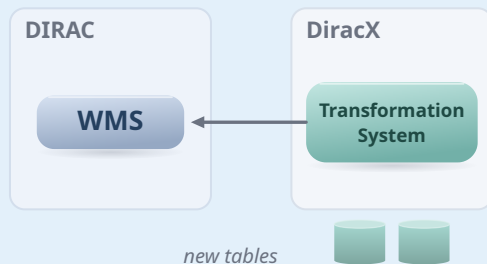


- **Scrum since 21 January 2026:** two-week sprints; planning and refinement in one weekly meeting; a monthly meeting with the representatives (DOPs); hackathons.
- **Large changes start with a proof of concept, then an ADR** that the other developers approve. ADRs and roadmap are public; the board is not.
- **We are moving towards this process; it is not there yet.** Like the roadmap, it adapts as we learn.

Not a port: a reimplementation

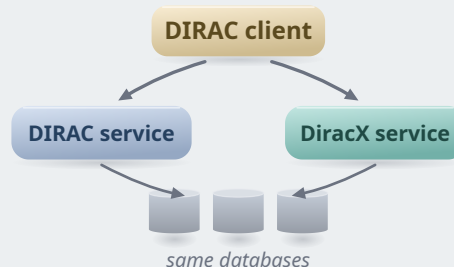
A new data model

The spine: workgraph, transformations, parcels. New requirements need new tables and new logic. The RMS is used as it is, as a backend.



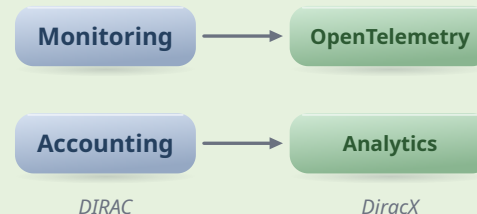
Future clients

DMS: a DIRAC client calls DiracX instead of DIRAC, over the same databases. **RSS:** the read side first, through an adapter.



Standard protocols

Systems internal to DIRAC give way to standards: monitoring to OpenTelemetry, accounting to analytics. Details later.



| 4 · Results so far

Numbers since the last workshop



Activity since the last workshop

17 SEP 2025 – 4 SEP 2026	DIRAC STACK	DIRACX STACK	BOTH
Merged pull requests	367	261	634
People who merged one	20	28	36
People who gave a first review	12	11	17
New contributors	9	14	17
Issues opened	89	206	296
Issues closed (completed / not planned)	117 (76 / 41)	224 (191 / 33)	343 (268 / 75)

DIRAC stack: DIRAC, WebAppDIRAC, DIRACOS2, Pilot. DiracX stack: diracx, diracx-charts, diracx-web, signurlarity, dirac-cwl, interCEde. Bots excluded; copies of one change on several release branches counted once. Issues to 2 Sep.

Since Scrum: the backlog is shrinking

295 → 195

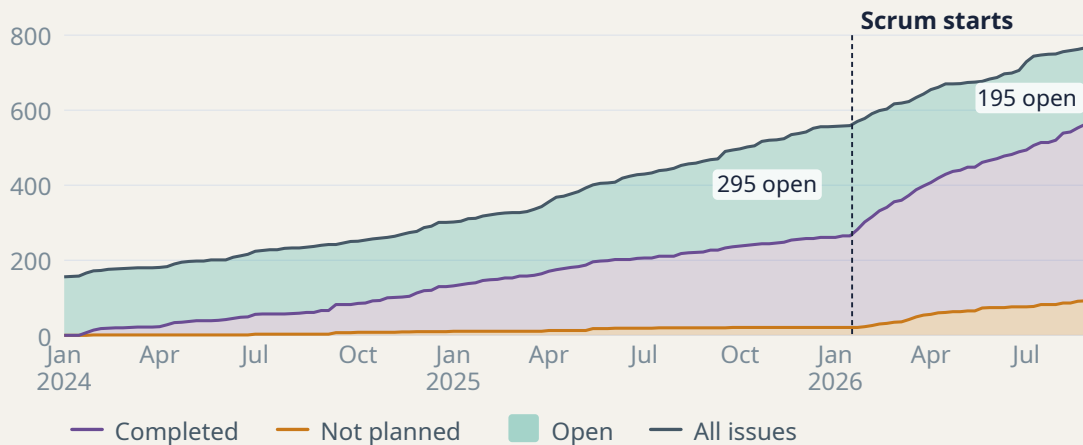
open issues, January 2026 to September

10.6 → 36.6

closures a month; 26.6 completed, 10.0 not planned or duplicate

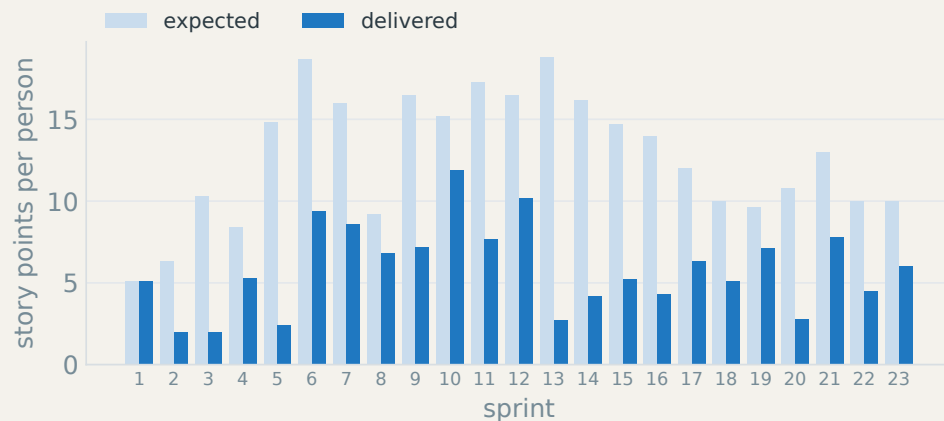
16.3 → 27.3

issues opened a month



All issues in the DIRACGrid repositories, bots excluded. Part of the fall is triage.

Velocity



- **We expect about twice what we deliver:** a median of 13 story points per person and sprint expected, 5.3 delivered.
- Delivered velocity follows availability: workshops, conferences, holidays.
- Only one sprint delivered what was expected.

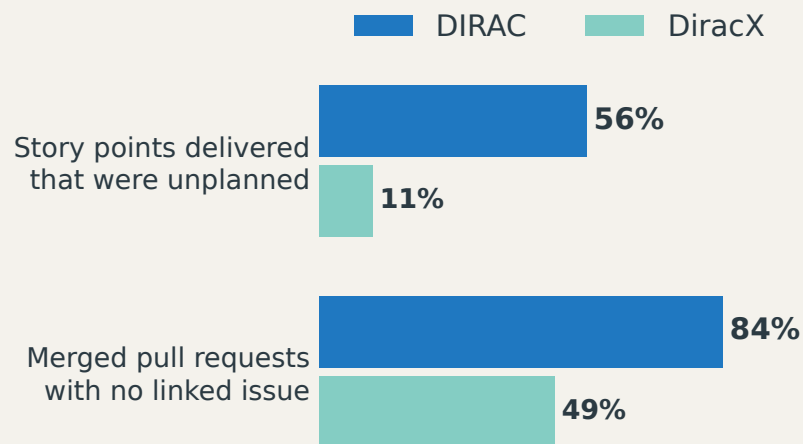
Where an issue goes



issues on the planning board, data to 4 September 2026

- Issues that move on do so quickly. The ones still waiting have waited far longer.
- 121 issues sit in triage, most since the January board sweep. 34 are "in progress" with a median of two months.

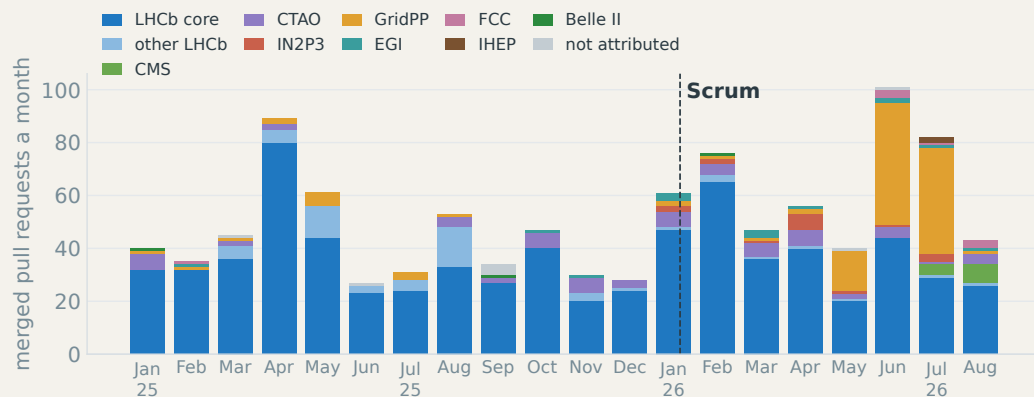
Unplanned work lives in DIRAC



- Since January, **56% of the story points delivered on DIRAC were unplanned**, against 11% on DiracX.
- **84% of DIRAC pull requests have no linked issue**, against 49% on DiracX.
- Security and performance work arrives when it arrives. It is the cost of keeping DIRAC running, and why DIRAC only gets patches.

For developers: we overestimate what a sprint can hold because we do not count the unplanned DIRAC maintenance. Plan for it.

More hands, from more communities

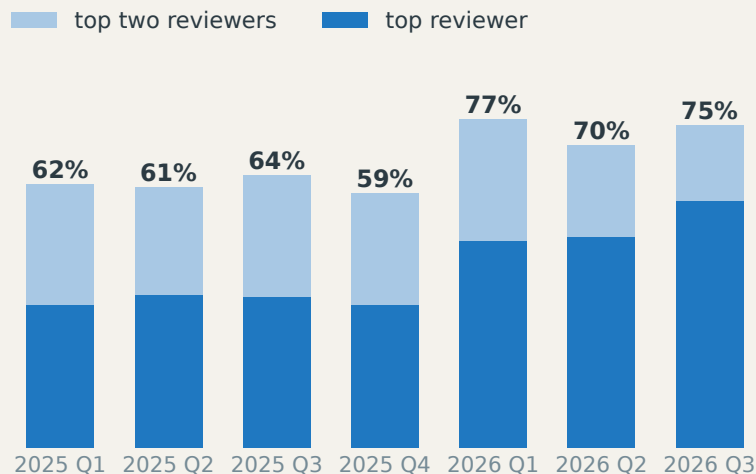


36 people merged a change since the last workshop; **17 were new**, from ten communities. Almost all are part-time, so effort varies from month to month.

Outside the five core developers: **36% of merged PRs, 28% of issues opened, 53% of story points.**

Read with care: about half of the 2026 rise in PRs is **one GridPP developer** hardening DIRAC's SQL. CMS merged 14 changes since July.

The bottleneck is review



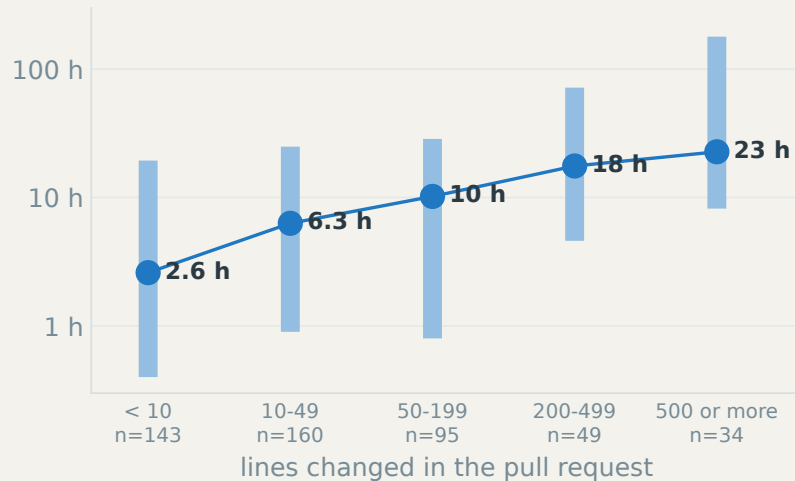
Share of first reviews, by quarter

- Two people give three quarters of first reviews. The top reviewer alone went from a third to over half.
- None of the 14 contributors new since January has given a first review yet.
- Design review is not in these numbers. ADRs are reviewed by everyone, in meetings and on chat.

→ *writing code has spread; reading it has not*

Question for developers: kanban style, should you review before starting something new once your tasks are done?

Smaller pull requests get reviewed sooner



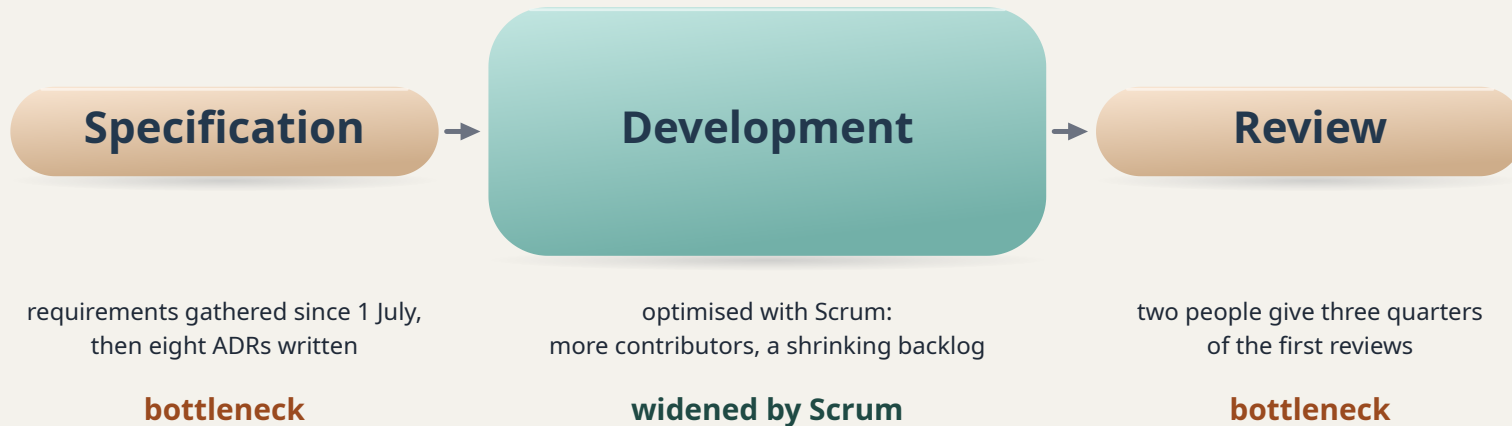
Median wait for a first review, with the middle half of the pull requests

- Under 10 lines: 2.6 hours. 500 lines or more: 23 hours. Time to merge goes from a few hours to five days.
- The spread is wide at every size, so size helps but does not decide.
- Who writes it matters too. For pull requests under 50 lines, the five core developers wait 2.5 hours; everyone else waits 11.

→ *split the work, and announce a big change early*

How to split a large change into stacked pull requests: [DiracX contributing guide](#)

Where work waits



- **Scrum widened development:** more contributors, a shrinking backlog.
- **Specification and review are the narrow parts,** and they will matter more with every new component.

For everyone in the room: representatives, developers and users, where would you improve how we work?

Cheaper code: easier to explore, same bottlenecks

"AI does not resolve ambiguity. It amplifies it."

Nir Yechiel, [The bottleneck moved](#), July 2026

- Writing code got cheap, so **spikes and proofs of concept** are easier to try: matchmaking, interCEde, analytics, observability.
- **Deciding what to build and checking it did** not get cheaper.
- It does **not change our process**. It shows its weak points sooner.

Each one is something we just saw

- **Vague specification** → verbose, AI-generated issues that go stale (an open action). A clear ADR with a scenario test is what an agent, or a newcomer, can implement.
- **Review bottleneck** → more code means more to read, by the same two people.
- **Unplanned work** → faster code does not shrink urgent maintenance.
- **A new kind of user** → DiracX speaks HTTP and OAuth2, so an agent can use it through MCP.

→ *invest in specifications, tests and review, not in typing*

| 5 · What's next

How we could improve, and the roadmap



Next: building the transformation system

Implementation starts at the hackathon this afternoon. The technical details are in the next talk.

Where it comes from

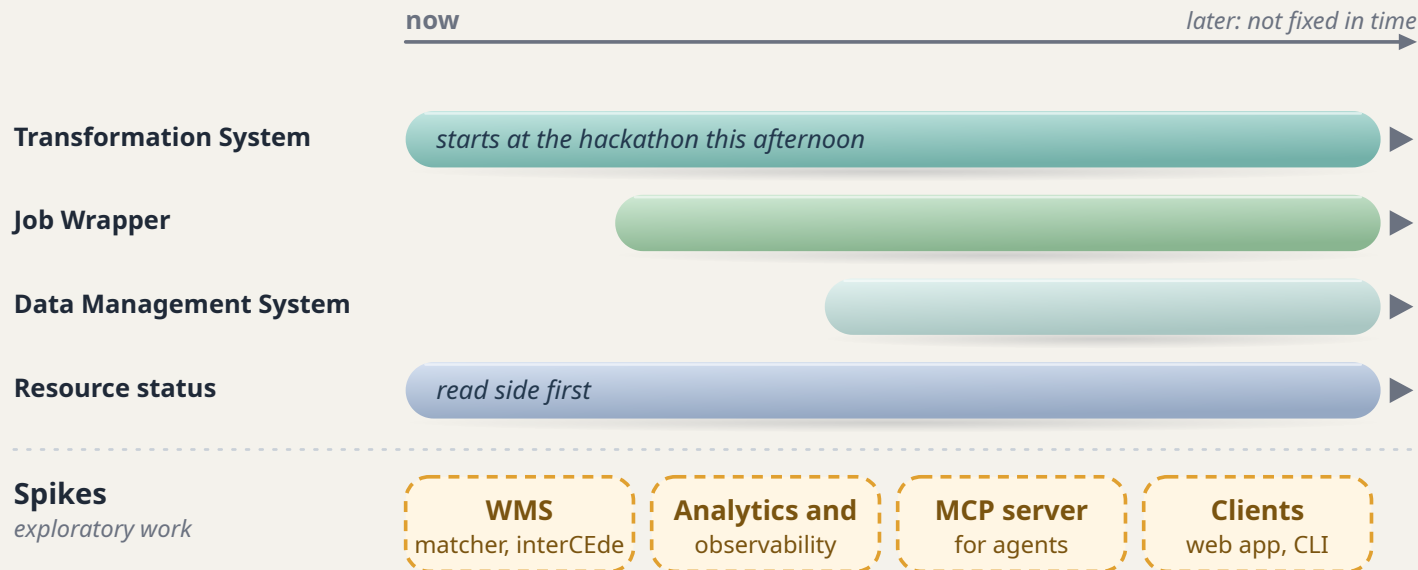
- **Use cases from the communities:** CMS, CTAO, FCC, Belle II and LHCb.
- **A design that comes from a proof of concept**, written down as ADRs.

How it will be delivered

- **Increments the representatives can test**, shown at DOps whenever a sprint delivers one.
- **The more we are, and the better organised, the sooner.**

This afternoon, two hackathons: adapting your community's workflows to **CWL**, and building the **transformation system**.

The roadmap moves with the communities



Priorities, not commitments. New requirements can reorder this page, and the team writes the detailed plan.

If you run an installation today

What you can count on

- **DIRAC keeps running**, with security patches and performance fixes when needed.
- **You choose when to switch each component**, usually at a shutdown.
- **No data migration** for anything but the transformation system, which has its own tables.
- **Your extension keeps working** on DIRAC while you move.
- **DIRAC and DiracX live in parallel**, and work can move gradually from one to the other.
- **Increments to test**, shown at DOpS whenever a sprint delivers one.

Take-aways

FOR INSTALLATIONS

- **Adopt what exists; DIRAC is only patched.** You move one component at a time, and the transformation system runs in parallel.

FOR REPRESENTATIVES AND DEVELOPERS

- **Specification and review are the bottlenecks.** Increments you can test, shown at DOps, keep us close to your needs.

FOR THIS AFTERNOON

- **Join one of the two hackathons:** adapting your workflows to CWL, or building the transformation system.

Thank you

Questions and comments welcome, this week and after.

Alexandre F. Boyer · alexandre.boyer@cern.ch

github.com/DIRACGrid · diracx.diracgrid.org

